UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACION ORIENTADA A OBJETOS

DIEGO ISAIAS MARROQUIN MOSCOSO

EJERCICIO #5

Análisis

Defensa de QTZ2: análisis y diseño de un simulador de misión espacial con herencia, polimorfismo y el patrón MVC.

1. ¿Qué problema resuelve el programa?

El prototipo administra los recursos de Quetzal-2 mediante módulos de vuelo, tierra y energía. Permite consultar componentes, buscarlos, ordenar un catálogo por costo, avanzar ciclos y revisar las comunicaciones. El alcance es una simulación por consola: los generadores abastecen energía, los instrumentos recolectan información y las antenas descargan datos. No se incorporan baterías ni mecánicas de enemigos o daño que el enunciado no define.

2. ¿Qué clases representan el problema?

Se utilizan ocho clases: Modulo, ModuloVuelo, ModuloTierra, ModuloEnergia, Mision, SimuladorController, SimuladorView y Main. Modulo es la superclase abstracta; las tres subclases implementan sus operaciones particulares. Mision conserva los recursos y la lista de componentes. El controlador coordina las opciones, la vista interactúa con el usuario y Main construye la carga inicial.

3. ¿Qué información debe conservarse?

Cada módulo tiene ID, nombre, salud, estado activo y costo de construcción. Vuelo conserva instrumento, datos por ciclo, consumo energético, ciclos exitosos y total recolectado. Tierra conserva estación, capacidad, energía por descarga e historial descargado. Energía conserva producción por ciclo, ciclos exitosos y total generado. Mision registra energía disponible, datos en órbita, datos descargados y ciclos transcurridos.

4. ¿Qué tipos y visibilidad tienen los datos?

Los identificadores y contadores son int; los nombres, estaciones e instrumentos son String; activo es boolean. Salud, costos, energía, capacidades y datos son double. Los atributos comunes de Modulo son protected; los particulares de las subclases, de Mision, de la vista y del controlador son private. Los constructores y operaciones son public. main() es static. Las firmas y el propósito de los miembros se detallan más adelante.

5. ¿Cómo se organiza la lista polimórfica?

Mision declara una única List<Modulo>, creada mediante ArrayList. Allí conviven las tres subclases y se conserva el orden de registro. agregarModulo(Modulo modulo) acepta cualquier subtipo. El arreglo Modulo[] de Main solo sirve para cargar esos objetos en la lista de Mision. Las listas auxiliares de catálogo y reportes son consultas, no nuevas colecciones que controlen el procesamiento.

6. ¿Cómo se aplica el polimorfismo vía herencia?

Modulo declara public abstract String procesarCiclo(Mision mision). Las tres subclases sobrescriben exactamente esa firma con @Override. Mision.avanzarCiclo() recorre referencias Modulo e invoca

modulo.procesarCiclo(this); Java selecciona la implementación del tipo real. No se usa instanceof para decidir cómo ejecutar un ciclo. El parámetro Mision permite actualizar los mismos recursos compartidos.

7. ¿Cómo opera un módulo de vuelo?

Si está activo y consumirEnergia(consumoEnergetico) tiene éxito, agrega todos sus datosPorCiclo a la órbita, incrementa ciclosAcumulados y suma al totalAcumulado. Si está inactivo o falta energía, conserva los recursos y contadores y devuelve un mensaje con la causa. No realiza recolecciones parciales. El total histórico no se reduce cuando una antena descarga esos datos.

8. ¿Cómo opera un módulo de tierra?

Primero comprueba actividad, energía suficiente y datos pendientes. Descarga el menor valor entre capacidadDescarga y datosEnOrbita; consume la energía completa por descarga aunque reciba menos datos que su capacidad. Reduce lo pendiente en órbita y aumenta tanto el total de la misión como totalDescargado del módulo. Si alguna condición falla, no consume energía ni descarga datos.

9. ¿Cómo opera un módulo de energía?

Si está activo, agrega energiaPorCiclo a Mision, incrementa ciclosAcumulados y suma al totalGenerado. Si está inactivo, informa la causa sin modificar esos valores. La producción y las capacidades por ciclo se fijan en los constructores y no tienen setters. No hay baterías ni límite de almacenamiento de energía definido en esta versión.

10. ¿Cómo transcurre un ciclo completo?

Mision incrementa ciclosTranscurridos una vez y ofrece un turno a todos los módulos en el orden de la carga inicial. Cada operación actualiza inmediatamente los recursos que usará el siguiente componente. Un módulo que no puede operar devuelve su explicación y el recorrido continúa. Los contadores particulares solo aumentan cuando corresponde, aunque el ciclo global avance. El controlador entrega los mensajes y el estado final a la vista.

11. ¿Cómo se aplican las responsabilidades de MVC?

El modelo está formado por Mision y la jerarquía Modulo; contiene estado y reglas, sin Scanner ni impresión de consola. SimuladorView lee entradas, valida su formato y presenta resultados. SimuladorController interpreta las opciones y conecta consultas u operaciones del modelo con la vista. Main configura objetos e inicia el controlador. La vista consulta getters y resultados ya calculados al mostrar estados y comunicaciones.

12. ¿Cómo funcionan las búsquedas y el catálogo?

buscarModuloPorId() compara el ID; buscarModuloPorNombre() compara nombres con equalsIgnoreCase(). Ambas devuelven Modulo o null. La carga usa IDs diferentes. obtenerCatalogoPorCosto() copia las referencias a una nueva ArrayList y ordena esa copia de menor a mayor costo mediante intercambio de elementos. La lista original permanece igual para que consultar el catálogo no altere los ciclos.

13. ¿Qué incluye el resumen de comunicaciones?

Muestra cuántos módulos de tierra existen y cuántos están activos, la suma de capacidades de los activos, el total histórico descargado por la misión y el nombre, ID y estación de todos los módulos empatados en el máximo histórico. El máximo considera también los inactivos. Si todos tienen cero, todos empatan; si no hay módulos de tierra, la lista de mayores descargadores queda vacía.

14. ¿Cómo se inicializa la misión?

Main crea Mision(500), diez módulos y los registra en el orden escrito. Los módulos ya existen y tienen distintos costos, salud y actividad; los historiales y ciclos comienzan en cero, igual que los datos en órbita y descargados. Luego crea SimuladorView, inyecta misión y vista en SimuladorController y llama a iniciarSimulador(). Este es el punto de partida elegido para representar una misión con infraestructura desplegada.

15. ¿Qué validaciones y decisiones de encapsulación utiliza el programa?

La vista solicita de nuevo textos vacíos, números de formato incorrecto, enteros negativos y opciones fuera de 1 a 6. Las condiciones operativas se verifican en cada módulo antes de consumir recursos. Los constructores reciben datos válidos definidos en Main. getModulos() devuelve la lista interna, por lo que el código que la recibe debe evitar modificarla; una vista de solo lectura sería una mejora de encapsulación, no una característica ya implementada.

16. ¿Cómo se podría ampliar el diseño?

Una nueva categoría extendería Modulo, inicializaría sus atributos con super(...) y sobrescribiría procesarCiclo(Mision) y toString(). El recorrido de Mision no cambiaría. Main tendría que

incorporar sus instancias y el análisis de comunicaciones solo requeriría cambios si la nueva categoría participara en ese reporte. equals() no se sobrescribe porque las búsquedas actuales comparan ID o nombre explícitamente; no se necesita inventar una igualdad adicional.

Requisitos funcionales del sistema

• Mantener una única lista polimórfica con al menos diez módulos de los tres tipos.

• Conservar y mostrar los atributos comunes y las características particulares de cada módulo.

• Listar todos los módulos construidos y buscar por ID o nombre, informando si no existe coincidencia.

• Presentar un catálogo por costo de construcción ascendente sin reordenar la lista operativa.

• Procesar cada ciclo en el orden de la carga y aplicar las actualizaciones de recursos inmediatamente.

• Recolectar todos los datos de un módulo de vuelo solo cuando esté activo y tenga energía suficiente.

• Descargar hasta la capacidad del módulo de tierra, pagando el consumo completo aun en una descarga parcial.

• Generar energía fija por ciclo únicamente con los módulos de energía activos.

• Conservar recursos e historiales si un módulo no puede operar y comunicar el motivo sin detener el recorrido.

• Acumular ciclos exitosos e historiales individuales y mantener los recursos entre ciclos sucesivos.

• Mostrar al terminar cada ciclo los resultados individuales y el estado de la misión.

• Consultar totales, activos, capacidad activa, histórico descargado y todos los máximos empatados de comunicaciones.

• Mantener el menú de consola hasta seleccionar salir y separar responsabilidades mediante MVC.

Carga inicial de la misión

Estado inicial: energía 500 unidades; datos en órbita 0 MB; descargados 0 MB; ciclos de misión 0. Los historiales de cada módulo y los ciclos acumulados de vuelo y energía empiezan en 0. El orden de los siguientes registros determina el orden de ejecución.

1. ID 8, Panel Solar Alpha: energía; salud 100; activo; costo 22000; genera 20 unidades por ciclo.

2. ID 1, Camara Orbital: vuelo; salud 90; activo; costo 50000; instrumento Cámara; 20 MB por ciclo; consume 5 unidades.

3. ID 5, Antena Central: tierra; salud 100; activo; costo 17000; Campus Central; capacidad 25 MB; consume 6 unidades por descarga.

4. ID 2, Sensor solar: vuelo; salud 80; activo; costo 12000; instrumento Sensor; 15 MB por ciclo; consume 5 unidades.

5. ID 9, Panel Solar Beta: energía; salud 80; activo; costo 16000; genera 15 unidades por ciclo.

6. ID 6, Antena Norte: tierra; salud 90; activo; costo 13000; Estacion Norte; capacidad 15 MB; consume 4 unidades por descarga.

7. ID 3, Camara Cientifica: vuelo; salud 100; inactivo; costo 5000; instrumento Cámara; 14 MB por ciclo; consume 4 unidades.

8. ID 7, Antena Sur: tierra; salud 75; inactivo; costo 19000; Estacion Sur; capacidad 10 MB; consume 3 unidades por descarga.

9. ID 10, Panel Solar Gamma: energía; salud 60; inactivo; costo 9000; genera 10 unidades por ciclo.

10. ID 4, Sensor Espacial: vuelo; salud 60; activo; costo 11000; instrumento Sensor; 21 MB por ciclo; consume 6 unidades.

La salud se almacena y se muestra, pero no disminuye automáticamente ni modifica la capacidad. El enunciado no define fórmulas de daño, reparación o reducción del rendimiento; el estado activo es la condición de habilitación usada por los módulos.

Descripción de las clases

Notación: + público; - privado; # protegido. Se muestran los miembros declarados, con parámetros y resultado. Los constructores no declaran retorno; {abstract} identifica el contrato abstracto y {static} el punto de entrada. Los miembros heredados no se duplican en las subclases.

Modulo {abstract}

Propósito de la clase:

Superclase abstracta que concentra los datos comunes, getters, setters de estado y el contrato de procesamiento. No se instancia directamente.

Propósito de los atributos:

id: int. Identificador numérico del componente.

nombre: String. Nombre usado en búsquedas y presentación.

salud: double. Estado de salud almacenado del módulo.

activo: boolean. Indica si está habilitado para operar.

costoConstruccion: double. Valor utilizado para ordenar el catálogo.

Propósito de los métodos:

+ Modulo(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double). Asigna los cinco datos comunes. Lo invocan las subclases mediante super(...).

+ getId(): int. Retorna id.

+ getNombre(): String. Retorna nombre.

+ getSalud(): double. Retorna salud.

+ isActivo(): boolean. Retorna activo.

+ getCostoConstruccion(): double. Retorna costoConstruccion.

+ setSalud(salud: double): void. Reemplaza el valor de salud recibido; no aplica una fórmula de daño.

+ setActivo(activo: boolean): void. Actualiza si el módulo está habilitado.

+ procesarCiclo(mision: Mision): String {abstract}. Contrato abstracto: recibe la misión compartida y debe retornar un mensaje con el resultado del turno.

+ toString(): String. Retorna una descripción con los atributos comunes.

ModuloVuelo

Propósito de la clase:

Consume energía para recolectar datos completos y conservar su historial científico.

Propósito de los atributos:

- tipoInstrumento: String. Cámara o sensor del módulo de vuelo.

- datosPorCiclo: double. MB recolectados en una operación completa.

- consumoEnergetico: double. Energía requerida por operación de vuelo.

- ciclosAcumulados: int. Cuenta únicamente operaciones realizadas.

- totalAcumulado: double. MB históricos recolectados por el módulo.

Propósito de los métodos:

+ ModuloVuelo(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, tipoInstrumento: String, datosPorCiclo: double, consumoEnergetico: double). Inicializa los datos comunes y específicos; ciclosAcumulados=0 y totalAcumulado=0.0.

+ getTipoInstrumento(): String. Retorna tipoInstrumento.

+ getDatosPorCiclo(): double. Retorna datosPorCiclo.

+ getConsumoEnergetico(): double. Retorna consumoEnergetico.

+ getCiclosAcumulados(): int. Retorna ciclosAcumulados.

+ getTotalAcumulado(): double. Retorna totalAcumulado.

+ procesarCiclo(mision: Mision): String. Sobrescribe el contrato; consume energía y recolecta datos completos solo cuando puede operar.

+ toString(): String. Sobrescribe toString(), reutiliza super.toString() y añade los atributos particulares e historiales.

ModuloTierra

Propósito de la clase:

Recibe datos de la órbita y registra sus descargas históricas y su estación.

Propósito de los atributos:

- estacion: String. Estación a la que pertenece la antena.

- capacidadDescarga: double. Límite de MB recibidos por ciclo.

- energiaPorDescarga: double. Consumo completo de cada descarga realizada.

- totalDescargado: double. MB históricos descargados por la antena.

Propósito de los métodos:

+ ModuloTierra(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, estacion: String, capacidadDescarga: double, energiaPorDescarga: double). Inicializa los datos comunes y específicos; totalDescargado=0.0.

+ getEstacion(): String. Retorna estacion.

+ getCapacidadDescarga(): double. Retorna capacidadDescarga.

+ getEnergiaPorDescarga(): double. Retorna energiaPorDescarga.

+ getTotalDescargado(): double. Retorna totalDescargado.

+ procesarCiclo(mision: Mision): String. Sobrescribe el contrato; valida actividad, energía y datos, y realiza una descarga completa o parcial.

+ toString(): String. Sobrescribe toString(), reutiliza super.toString() y añade los atributos particulares e historiales.

ModuloEnergia

Propósito de la clase:

Genera energía y registra producción histórica y ciclos efectivamente operados.

Propósito de los atributos:

- energiaPorCiclo: double. Producción fija por ciclo exitoso.

- ciclosAcumulados: int. Cuenta únicamente operaciones realizadas.

- totalGenerado: double. Suma histórica de energía producida.

Propósito de los métodos:

+ ModuloEnergia(id: int, nombre: String, salud: double, activo: boolean, costoConstruccion: double, energiaPorCiclo: double). Inicializa los datos comunes y energiaPorCiclo; ciclosAcumulados=0 y totalGenerado=0.

+ getEnergiaPorCiclo(): double. Retorna energiaPorCiclo.

+ getCiclosAcumulados(): int. Retorna ciclosAcumulados.

+ getTotalGenerado(): double. Retorna totalGenerado.

+ procesarCiclo(mision: Mision): String. Sobrescribe el contrato; genera energía solo si está activo.

+ toString(): String. Sobrescribe toString(), reutiliza super.toString() y añade los atributos particulares e historiales.

Mision

Propósito de la clase:

Conserva la lista y recursos compartidos, recorre módulos polimórficamente y calcula consultas y reportes.

Propósito de los atributos:

- modulos: List<Modulo>. Lista única de referencias polimórficas en orden de participación.

- energiaDisponible: double. Unidades de energía actualmente disponibles.

- datosEnOrbita: double. MB pendientes de descarga.

- datosDescargados: double. MB descargados históricamente por toda la misión.

- ciclosTranscurridos: int. Número de ciclos globales solicitados.

Propósito de los métodos:

+ Mision(energiaDisponible: double). Crea una ArrayList vacía, asigna la energía recibida y deja datos y ciclos en cero.

+ getModulos(): List<Modulo>. Retorna la lista interna modulos.

+ getEnergiaDisponible(): double. Retorna energiaDisponible.

+ getDatosEnOrbita(): double. Retorna datosEnOrbita.

+ getDatosDescargados(): double. Retorna datosDescargados.

+ getCiclosTranscurridos(): int. Retorna ciclosTranscurridos.

+ agregarModulo(modulo: Modulo): void. Agrega la referencia al final de la lista; conserva el orden de carga.

+ agregarEnergia(cantidad: double): void. Suma la cantidad recibida a la energía disponible.

+ consumirEnergia(cantidad: double): boolean. Resta la cantidad y retorna true si alcanza la energía; si no, retorna false sin cambios.

+ agregarDatosEnOrbita(cantidad: double): void. Suma la cantidad de MB recolectados a los pendientes.

+ descargarDatos(cantidad: double): boolean. Transfiere la cantidad de órbita al total descargado y retorna true si existen suficientes datos; en otro caso retorna false.

+ buscarModuloPorId(id: int): Modulo. Retorna la primera referencia cuyo ID coincide o null si no existe.

+ buscarModuloPorNombre(nombre: String): Modulo. Retorna la primera coincidencia de nombre, ignorando mayúsculas, o null.

+ avanzarCiclo(): List<String>. Incrementa el ciclo global, llama procesarCiclo(this) en orden y retorna los mensajes.

+ obtenerModulosTierra(): List<ModuloTierra>. Retorna una lista auxiliar con los objetos de tipo ModuloTierra.

+ contarModulosTierra(): int. Retorna el número de módulos de tierra.

+ contarModulosTierraActivos(): int. Retorna el número de módulos de tierra habilitados.

+ obtenerCatalogoPorCosto(): List<Modulo>. Retorna una copia ordenada ascendentemente por costo sin alterar el orden original.

+ calcularCapacidadDescargaActiva(): double. Suma las capacidades de los módulos de tierra activos; no representa una garantía de descarga efectiva.

+ obtenerMayoresDescargadores(): List<ModuloTierra>. Retorna todos los módulos de tierra con el mayor total histórico, incluidos los empates; lista vacía si no hay antenas.

SimuladorController

Propósito de la clase:

Coordina las opciones del menú entre modelo y vista; no implementa las fórmulas de los módulos.

Propósito de los atributos:

- mision: Mision. Referencia al modelo compartido.

- view: SimuladorView. Referencia a la vista de consola.

Propósito de los métodos:

+ SimuladorController(mision: Mision, view: SimuladorView). Recibe y conserva referencias a la misma misión y vista.

+ listarModulos(): void. Obtiene la lista del modelo y solicita a la vista que la muestre.

+ consultarCatalogoPorCosto(): void. Obtiene el catálogo ordenado y lo entrega a la vista.

+ consultarResumenComunicaciones(): void. Solicita mostrar el resumen del modelo.

+ avanzarCiclo(): void. Solicita avanzar un ciclo al modelo y muestra resultados y estado mediante la vista.

+ buscarModulo(): void. Pide ID o Nombre, consulta el modelo y muestra el módulo o un aviso de ausencia.

+ iniciarSimulador(): void. Presenta la bienvenida y el menú de seis opciones en un ciclo; retorna al seleccionar salir.

SimuladorView

Propósito de la clase:

Gestiona la entrada y salida por consola y presenta datos del modelo.

Propósito de los atributos:

- scanner: Scanner. Lector de System.in.

Propósito de los métodos:

+ SimuladorView(). Inicializa scanner mediante new Scanner(System.in).

+ mostrarMenuPrincipal(): void. Imprime las seis opciones disponibles.

+ leerString(mensaje: String): String. Presenta el mensaje y retorna texto sin espacios extremos; repite si está vacío.

+ leerEntero(mensaje: String): int. Retorna un entero no negativo; captura errores de formato y repite la lectura.

+ leerDouble(mensaje: String): double. Retorna un double no negativo según las comprobaciones implementadas; no se utiliza en el menú actual.

+ leerOpcion(): int. Retorna una opción entera entre 1 y 6, repitiendo ante entradas inválidas.

+ mostrarMensaje(mensaje: String): void. Imprime el texto recibido.

+ mostrarModulo(modulo: Modulo): void. Imprime el objeto usando su toString() polimórfico.

+ mostrarModulos(modulos: List<Modulo>): void. Recorre e imprime las descripciones de la lista recibida.

+ mostrarCatalogo(modulos: List<Modulo>): void. Presenta la lista en el orden ya calculado por Mision.

+ mostrarResultadosCiclo(resultados: List<String>): void. Imprime cada mensaje producido durante el ciclo.

+ mostrarEstadoMision(mision: Mision): void. Consulta y muestra energía, órbita, descargas, ciclos y cantidad de módulos.

+ mostrarResumenComunicaciones(mision: Mision): void. Muestra conteos, capacidad activa, histórico y nombre, ID y estación de todos los máximos.

Main

Propósito de la clase:

Punto de entrada que crea e interconecta la misión, los módulos, la vista y el controlador.

Propósito de los atributos:

No declara atributos de instancia; sus referencias y el arreglo inicial son variables locales de main().

Propósito de los métodos:

+ main(args: String[]): void {static}. Construye Mision(500), registra diez módulos, crea vista y controlador y llama a iniciarSimulador(). No declara constructor explícito.

Relaciones y decisiones de diseño

• ModuloVuelo, ModuloTierra y ModuloEnergia heredan de Modulo. La generalización se representa con línea continua y triángulo vacío hacia la superclase.

• Mision conserva una asociación navegable hacia 0..* módulos. Los recibe desde Main; no se presupone propiedad exclusiva ni destrucción conjunta. Los módulos reciben Mision temporalmente como parámetro de procesarCiclo(), por lo que esa relación es una dependencia.

• SimuladorController conserva asociaciones hacia Mision y SimuladorView. Main depende de las clases concretas que construye. La vista depende de Mision, Modulo y ModuloTierra para presentar consultas.

• El polimorfismo también aparece en agregarModulo(Modulo), mostrarModulo(Modulo), las colecciones List<Modulo> y la llamada a toString(). Las consultas de comunicaciones usan instanceof para seleccionar módulos de tierra, sin reemplazar el despacho polimórfico del ciclo.

• No hay una clase por cada cámara o antena: son instancias con diferentes valores. El costo se usa para el catálogo; no se implementa compra o descuento de presupuesto porque el menú solicitado no define esas operaciones.

Verificación de las reglas del simulador

Se compilaron los ocho archivos Java y se ejecutaron 36 comprobaciones del modelo, todas satisfactorias. Además, se recorrió el menú de Main para revisar la carga inicial, dos ciclos y los reportes. Los siguientes resultados distinguen el ejemplo del enunciado de la carga de diez módulos del programa.

Ejemplo de tres módulos del PDF

Con 10 unidades iniciales: el generador agrega 20 y deja 30; vuelo consume 8 y agrega 15 MB, dejando 22; tierra consume 5 y descarga 10 MB. Esperado y obtenido: 17 unidades, 5 MB en órbita y 10 MB descargados. En un segundo ciclo: 24 unidades, 10 MB en órbita y 20 MB históricos; los valores no se reinician.

Inactividad y recursos insuficientes

Un vuelo que requiere 8 unidades, al recibir una misión con 7, no consume, no recolecta y no aumenta ciclos. Un generador inactivo tampoco genera ni incrementa su contador. Tierra sin datos

no consume energía; con datos pero energía insuficiente los conserva. Se verificaron los mismos valores antes y después de los rechazos.

Descarga parcial

Con 3 MB en órbita, energía 7, capacidad 10 MB y consumo 5, tierra descarga 3 MB y deja 2 unidades de energía, 0 MB pendientes y 3 MB descargados. Esperado y obtenido coinciden: la energía se cobra completa aunque la descarga sea menor que la capacidad.

Orden del ciclo y catálogo

Con tierra antes de vuelo, una misión de 10 unidades no descarga datos en el primer turno porque aún no existen. Luego vuelo consume 8 y recolecta 15 MB; al final energía agrega 20. Esperado y obtenido: 22 unidades, 15 MB pendientes y 0 descargados. Ordenar el catálogo devolvió el menor costo primero sin cambiar el primer módulo de la lista operativa.

Búsquedas y empates

Se verificaron búsquedas por ID y por nombre sin distinguir mayúsculas, y null para un ID inexistente. Con tres antenas de historial cero, el máximo incluye a las tres. Tras dos descargas iguales de 10 MB por las dos activas, el reporte devuelve ambas como máximas, capacidad activa 20 MB y total descargado 20 MB.

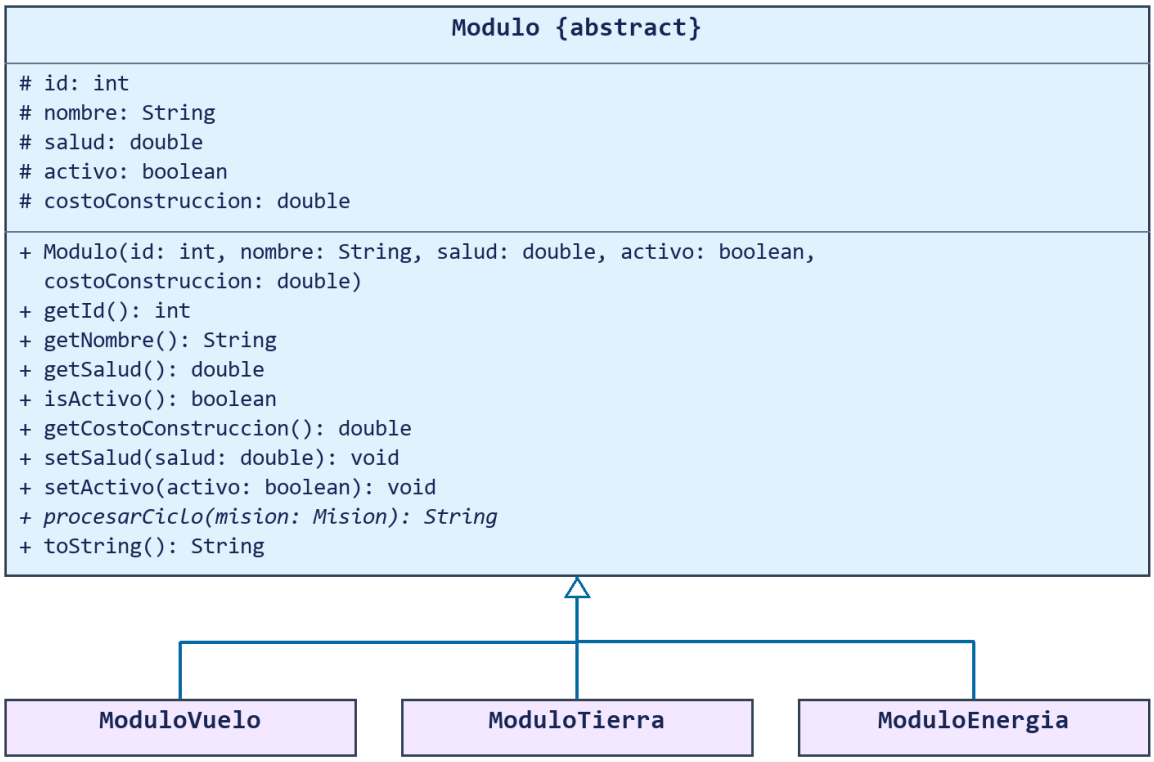
Carga real de Main

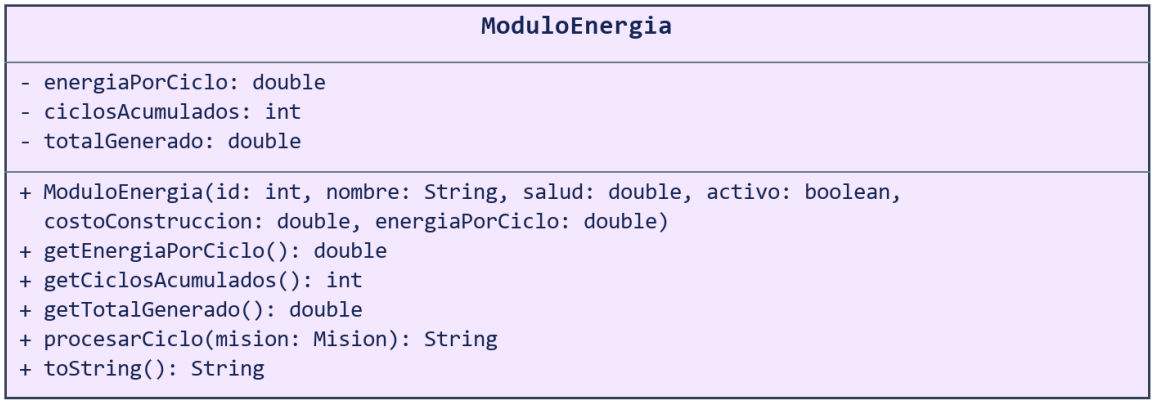
Inicialmente hay diez módulos, tres de tierra y dos antenas activas; capacidad activa 40 MB por ciclo. El primer ciclo deja 509 unidades, 21 MB en órbita y 35 MB descargados. El segundo deja 518 unidades, 37 MB en órbita y 75 MB descargados. Son los resultados esperados y obtenidos al ejecutar el programa.

Mayor descargador de la carga real

Al inicio Antena Central, Antena Norte y Antena Sur empatan en cero y aparecen las tres. Tras el primer ciclo, Antena Central acumula 20 MB y Antena Norte 15 MB. Tras el segundo, acumulan 45 y 30 MB. El resumen muestra como mayor a Antena Central, ID 5, estación Campus Central.

Diseño UML de la jerarquía

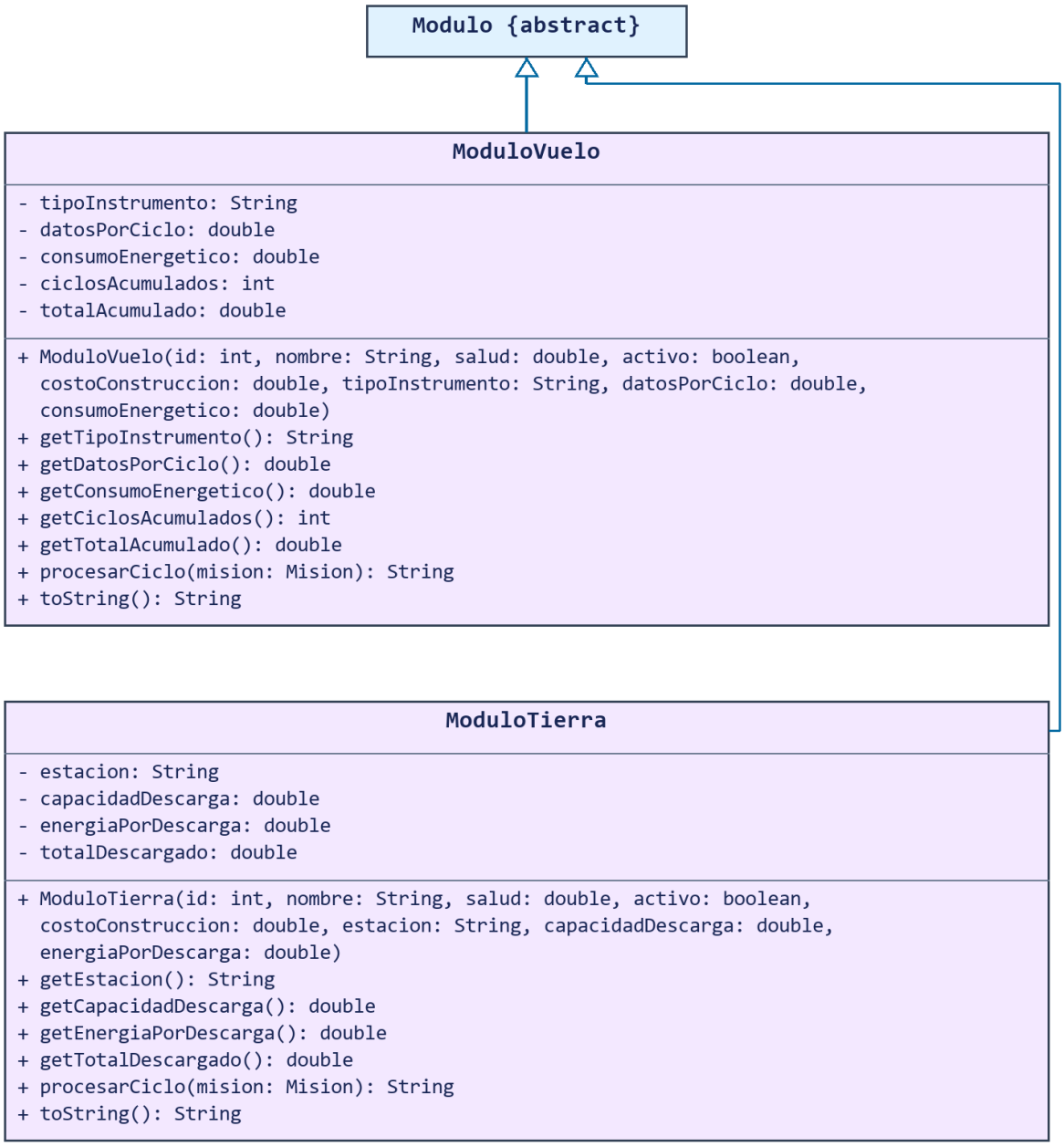




Generalización hacia Modulo; las tres subclases implementan procesarCiclo(Mision). El detalle de energía se incluye debajo de la jerarquía.

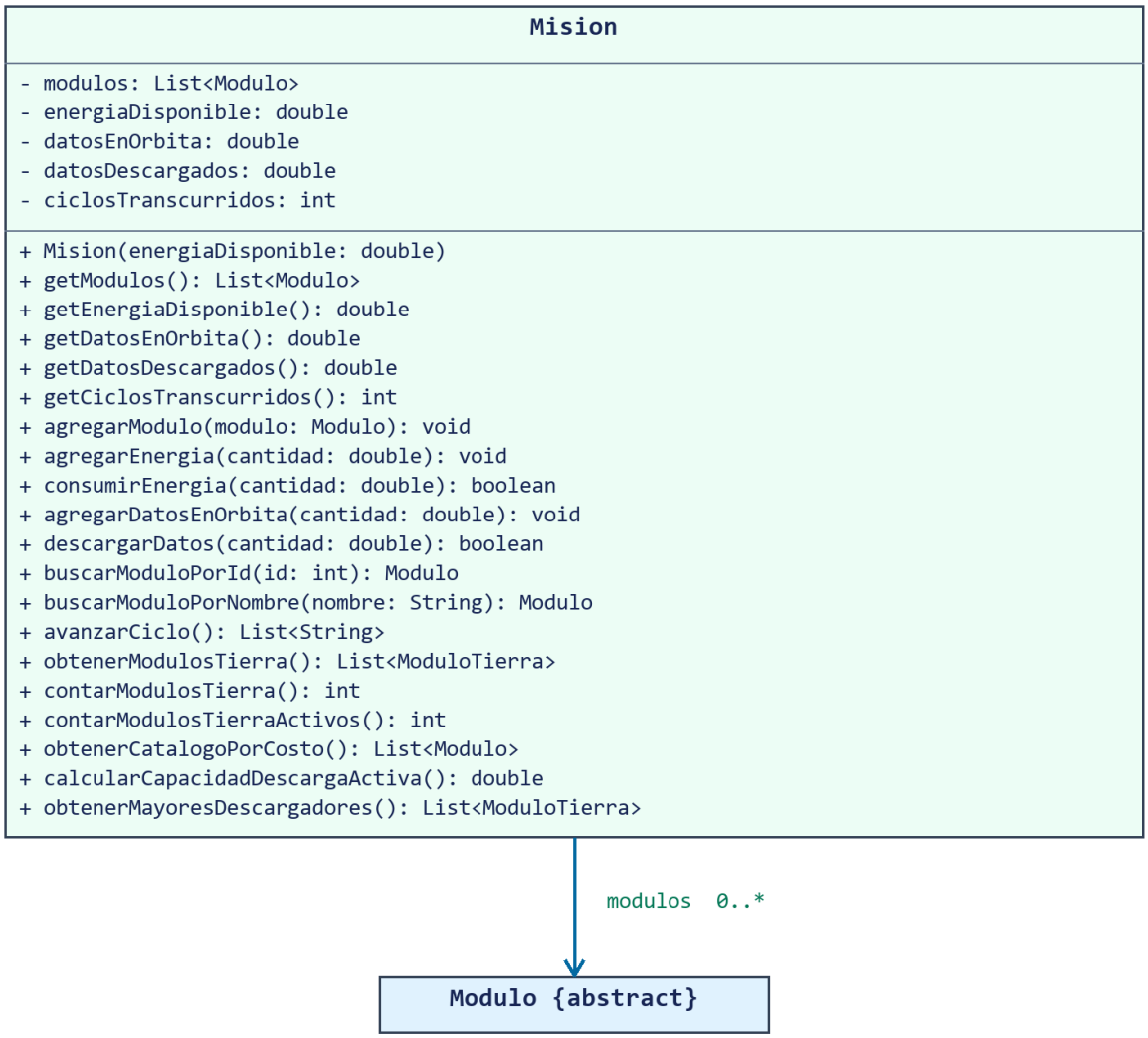
Visibilidad: + public, - private, # protected. Clase y operación abstractas se identifican con {abstract} o cursiva; main() aparece subrayado por ser static. Las cajas muestran atributos, constructores y métodos declarados.

Diseño UML de vuelo y tierra



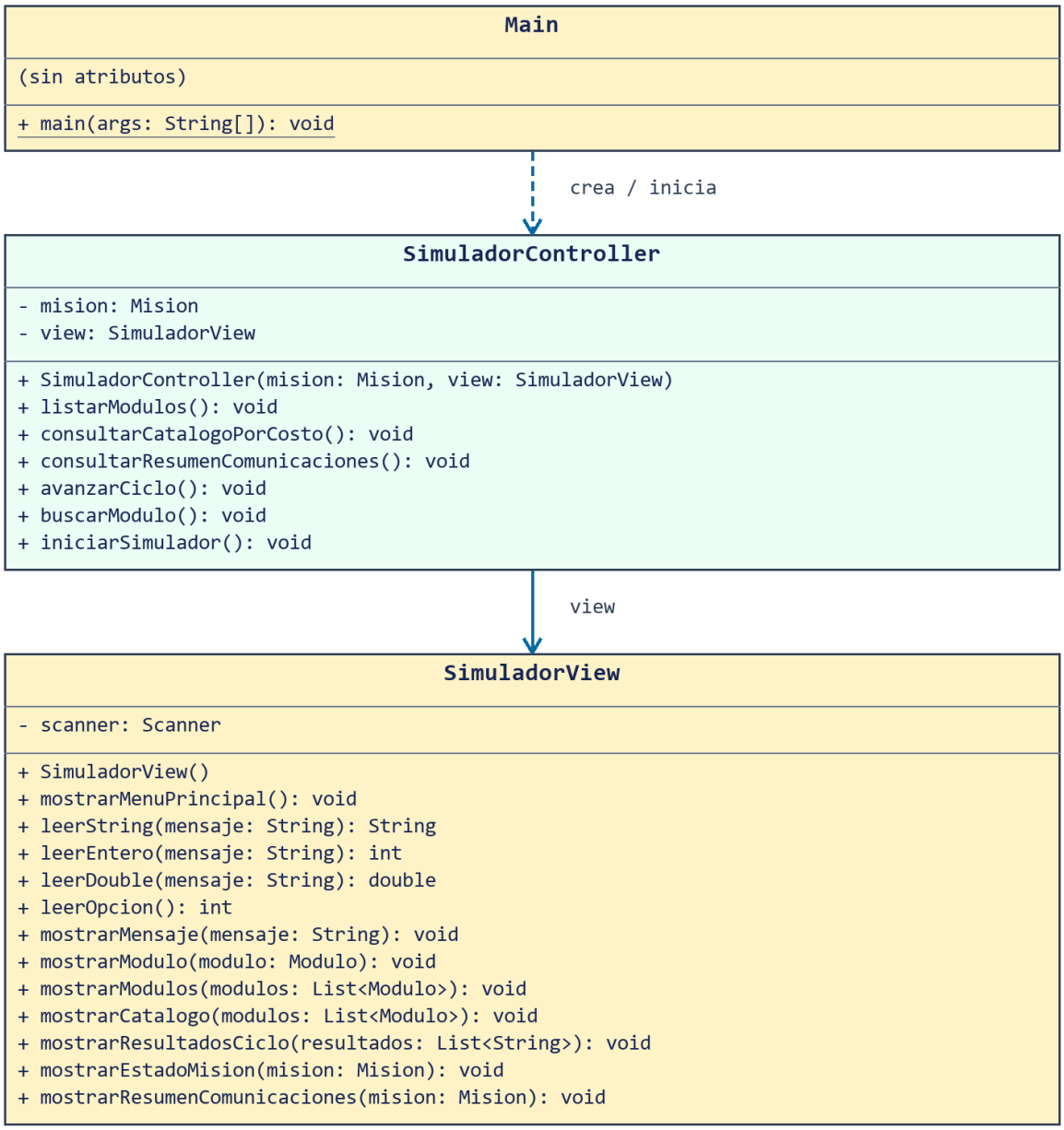
Detalle de las dos subclases. Las operaciones procesarCiclo() y toString() sobrescriben las declaradas en Modulo.

Diseño UML de la misión



Mision mantiene 0..* módulos y les delega el ciclo. Modulo y sus subclases dependen de Mision mediante el parámetro de procesarCiclo().

Diseño UML de MVC e inicio



Main crea los objetos e inicia el controlador. El controlador conserva referencias a Mision y SimuladorView; la vista consulta el modelo y muestra resultados.